

Resolución del problema Knapsack 0-1 con algoritmos genéticos

Maya Hristakeva
Departamento de Informática
Colegio Simpson
hristake@simpson.edu

Dipti Shrestha
Departamento de Informática
Colegio Simpson
shresthd@simpson.edu

Resumen

Este artículo describe un proyecto de investigación sobre el uso de Algoritmos Genéticos (AGs) para resolver el Problema de la Mochila 0-1 (KP). El Problema de la Mochila es un ejemplo de problema de optimización combinatoria, que busca maximizar el beneficio de los objetos de una mochila sin exceder su capacidad.

El artículo consta de tres secciones: breve descripción de la idea básica y los elementos de AG, definición del problema de la mochila y aplicación del problema de la mochila 0-1 mediante AG. El enfoque principal del artículo se centra en la implementación del algoritmo para resolver el problema. En el programa, implementamos dos funciones de selección, la ruleta y la selección de grupos. Los resultados de ambas difieren dependiendo de si utilizamos elitismo o no. El elitismo mejoró significativamente el rendimiento de la función de ruleta. Además, probamos el programa con diferentes proporciones de cruce y puntos de cruce simples y dobles, pero los resultados obtenidos no fueron muy diferentes.

Introducción

En este proyecto utilizamos Algoritmos Genéticos para resolver el problema 0-1 Knapsack donde hay que maximizar el beneficio de objetos en una mochila sin exceder su capacidad.

Dado que el problema Knapsack es un problema NP, enfoques como la programación dinámica, backtracking, branch and bound, etc. no son muy útiles para resolverlo. Los algoritmos genéticos los superan a todos y demuestran ser el mejor enfoque para obtener soluciones a problemas que tradicionalmente se consideraban inviables desde el punto de vista computacional, como el problema de Knapsack.

Algoritmos genéticos (AG)

Los algoritmos genéticos son algoritmos informáticos que buscan buenas soluciones a un problema entre un gran número de soluciones posibles. Fueron propuestos y desarrollados en la década de 1960 por John Holland, sus estudiantes y sus colegas de la Universidad de Michigan. Estos paradigmas computacionales se inspiraron en la mecánica de la evolución natural, incluida la supervivencia del más apto, la reproducción y la mutación. Estos mecanismos son muy adecuados para resolver una gran variedad de problemas prácticos, incluidos los problemas computacionales, en muchos campos. Algunas aplicaciones de los AG son la optimización, la programación automática, el aprendizaje automático, la economía, los sistemas inmunitarios, la genética de poblaciones y los sistemas sociales.

Idea básica de los AG

Los AG comienzan con un conjunto de soluciones candidatas (cromosomas) denominado población. Se crea una nueva población a partir de soluciones de una población antigua con la esperanza de obtener una población mejor. Las soluciones que se eligen para formar nuevas soluciones (descendientes) se seleccionan en función de su aptitud. Cuanto más aptas sean las soluciones, más posibilidades tendrán de reproducirse. Este proceso se repite hasta que se cumple alguna condición [1].

Elementos básicos de los AG

La mayoría de los métodos de AG se basan en los siguientes elementos: poblaciones de cromosomas, selección en función de la aptitud, cruce para producir nuevos descendientes y mutación aleatoria de los nuevos descendientes [2].

Cromosomas

Los cromosomas de los AG representan el espacio de soluciones candidatas. Las posibles codificaciones de los cromosomas son la binaria, la de permutación, la de valor y la de árbol. Para el problema Knapsack, utilizamos la codificación binaria, donde cada cromosoma es una cadena de bits, 0 o 1.

Función de aptitud

Los AG requieren una función de aptitud que asigna una puntuación a cada cromosoma de la población actual. Así, puede calcular lo bien codificadas que están las soluciones y lo bien que resuelven el problema [2].

Selección

El proceso de selección se basa en la aptitud. Los cromosomas evaluados con valores más altos (más aptos) serán seleccionados para reproducirse, mientras que los que tengan valores bajos serán descartados. Los cromosomas más aptos pueden ser seleccionados varias veces, pero el número de cromosomas seleccionados para reproducirse es igual al tamaño de la población, por lo que el tamaño se mantiene constante en cada generación. Esta fase tiene un elemento de aleatoriedad, al igual que la supervivencia de los organismos en la naturaleza. Los métodos de selección más utilizados son la ruleta, la selección por rangos, la selección en estado estacionario y otros. Además, para aumentar el rendimiento de los AG, los métodos de selección se mejoran mediante el elitismo. El elitismo es un método que primero copia algunos de los cromosomas con mejor puntuación en la nueva población y luego continúa generando el resto de la población. De este modo, se evita perder las pocas soluciones mejor encontradas.

Cruce

El cruce es el proceso de combinar los fragmentos de un cromosoma con los de otro. Con ello se pretende crear una descendencia para la siguiente generación que herede rasgos de ambos progenitores.

El cruce elige aleatoriamente un locus e intercambia las subsecuencias anterior y posterior a ese locus entre dos cromosomas para crear dos descendientes [2]. Por ejemplo, consideremos los siguientes progenitores y un punto de cruce en la posición 3:

Padre 1	1 0 0		0 1 1 1
Padre 2	1 1 1		1 0 0 0
Descendencia 1	1 0 0		1 0 0 0
Descendencia 2	1 1 1		0 1 1 1

En este ejemplo, el descendiente 1 hereda los bits de las posiciones 1, 2 y 3 del lado izquierdo del punto de cruce del padre 1 y el resto del lado derecho del punto de cruce del padre 2. De forma similar, el descendiente 2 hereda los bits de las posiciones 1, 2 y 3 del lado izquierdo del padre 2 y el resto del lado derecho del padre 1. Del mismo modo, el descendiente 2 hereda bits en las posiciones 1, 2 y 3 del lado izquierdo del padre 2 y el resto del lado derecho del padre 1.

Mutación

La mutación se realiza después del cruce para evitar que todas las soluciones de la población caigan en un óptimo local del problema resuelto. La mutación cambia la nueva descendencia volteando los bits de 1 a 0 o de 0 a 1. La mutación puede producirse en cada posición de bit de la cadena con

cierta probabilidad, normalmente muy pequeña (por ejemplo, 0,001). Por ejemplo, considere el siguiente cromosoma con punto de mutación en la posición 2:

Cromosoma no mutado:	1 0 0 0 1 1 1
Mutado:	1 1 0 0 1 1 1

El 0 en la posición 2 cambia a 1 después de la mutación.

Esquema de los AG s básicos

1. Inicio: Generar aleatoriamente una población de N cromosomas.
2. Aptitud: Calcula la aptitud de todos los cromosomas.
3. Crear una nueva población:
 - a. Selección: Según el método de selección seleccionar 2 cromosomas de la población.
 - b. Cruce: Realiza el cruce sobre los 2 cromosomas seleccionados.
 - c. Mutación: Realizar la mutación en los cromosomas obtenidos.
4. Sustituir: Sustituye la población actual por la nueva población.
5. Prueba: Comprueba si se cumple la condición final. En caso afirmativo, se detiene. En caso contrario, devuelve la mejor solución de la población actual y pasa a la etapa 2.

Cada iteración de este proceso se denomina generación.

El problema de la mochila (KP)

Definición

El problema KP es un ejemplo de problema de optimización combinatoria que busca la mejor entre muchas otras. Se trata de una mochila que tiene un volumen (o capacidad) V entero positivo. Hay n objetos distintos que pueden colocarse en la mochila. El artículo i tiene un volumen entero positivo V_i y un beneficio entero positivo B_i . Además, hay Q_i copias del artículo i disponibles, donde la cantidad Q_i es un entero positivo que satisface $1 \leq Q_i \leq \infty$.

X_i determina cuántas copias del artículo i se colocarán en la mochila. El objetivo es:

Maximice

$$\sum_{i=1}^N B_i X_i$$

Sujeto a las limitaciones

$$\sum_{i=1}^N V_i X_i \leq V$$

Y

$$0 \leq x_i \leq Q_i.$$

Si uno o más de los Q_i es infinito, el KP *no está acotado*; en caso contrario, el KP *está acotado*.

[3]. Los KP acotados pueden ser *KP 0-1* o *KP Multiconstraint*. Si $Q_i = 1$ para $i = 1, 2, \dots, N$, el problema es un *problema de mochila 0-1*. En el presente trabajo, hemos trabajado en el *problema de mochila 0-1* acotado, en el que no podemos tener más de una copia de un elemento en la mochila.

Ejemplo de 0-1 KP

Supongamos que tenemos una mochila con una capacidad de 13 pulgadas cúbicas y varios artículos de diferentes tamaños y beneficios. Queremos incluir en la mochila sólo los artículos que tengan el mayor beneficio total dentro de la capacidad de la mochila.

Hay tres partidas potenciales (etiquetadas como "A", "B" y "C"). Sus volúmenes y beneficios son los siguientes:

Artículo	A	B	C
Beneficio	4	3	5
Volumen	6	7	8

Buscamos maximizar el beneficio total:

$$\sum_{i=1}^3 B_i X_i = 4X_1 + 3X_2 + 5X_3$$

Sujeto a las :

$$\sum_{i=1}^3 V_i X_i = 6X_1 + 7X_2 + 8X_3 \leq 13$$

Y

$$X_i \in \{0,1\}, \text{ para } i = 1, 2, \dots, n.$$

Para este problema hay 2^3 posibles subconjuntos de elementos:

A	B	C	Volumen del	configure	Ventajas del set
0	0	0	0		0
0	0	1	8		5
0	1	0	7		3
0	1	1	15		-
1	0	0	6		4
1	0	1	14		-
1	1	0	13		7
1	1	1	21		-

Para encontrar la mejor solución tenemos que identificar un subconjunto que cumpla la restricción y tenga el máximo beneficio total. En nuestro caso, sólo las filas que aparecen en cursiva satisfacen la

. Por lo tanto, el beneficio óptimo para la restricción dada ($V = 13$) sólo puede obtenerse con una cantidad de A, una cantidad de B y cero cantidad de C, y es 7.

Problemas NP y el PK 0-1

Los problemas NP (polinómicos no deterministas) son aquellos para los que no existen algoritmos conocidos que garanticen su ejecución en un tiempo polinómico. Sin embargo, es posible "adivinar" una solución y, ambas cosas en tiempo polinómico. Algunos de los problemas NP más conocidos son el viajante de comercio, el circuito de Hamilton, el bin packing, el knapsack y el clique [4].

Los AG han demostrado ser muy adecuados para encontrar soluciones de alta calidad a problemas NP de gran envergadura y, en la actualidad, son las formas más eficientes de encontrar una solución aproximadamente óptima para problemas de optimización. No implican algoritmos de búsqueda extensivos y no intentan encontrar la mejor solución, sino que simplemente generan un candidato a solución, comprueban en tiempo polinómico si es una solución o no y lo buena que es. Los AG no siempre dan la solución óptima, sino una solución que se acerca bastante a la óptima.

Implementación del PK 0-1 mediante AGs

Representación de los artículos

Utilizamos una estructura de datos, llamada *celda*, con dos campos (beneficio y volumen) para representar cada elemento. A continuación, utilizamos una matriz de tipo *celda* para almacenar todos los elementos en ella, que tiene el siguiente aspecto:

artículos	0	1	2	3
	20 30	5 10	10 20	40 50

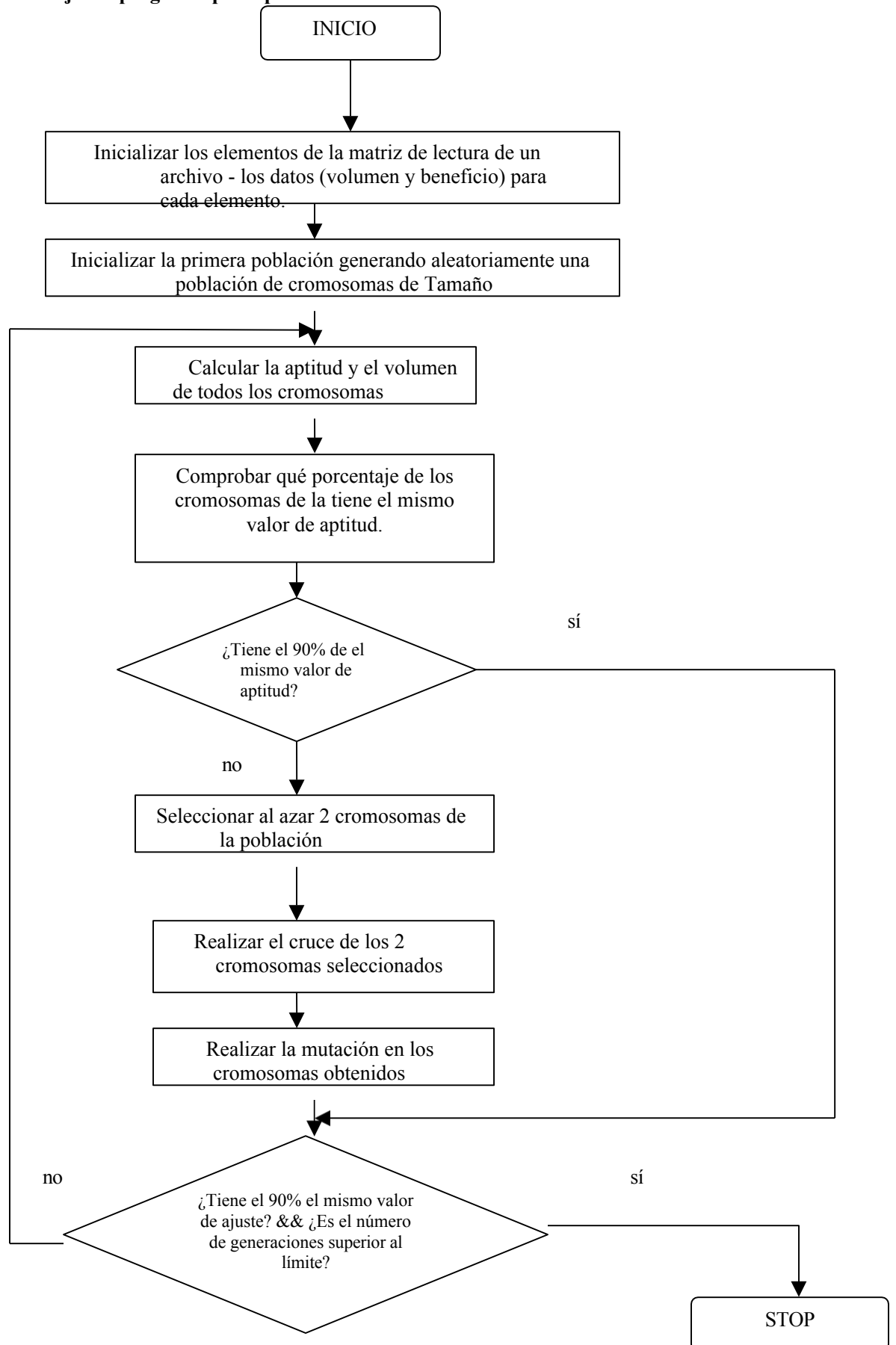
Codificación de los cromosomas

Un cromosoma puede representarse en una matriz de tamaño igual al número de elementos (en nuestro ejemplo, de tamaño 4). Cada elemento de esta matriz indica si un elemento está incluido en la mochila ('1') o no ('0'). Por ejemplo, el siguiente cromosoma:

0	1	2	3
1	0	0	1

indica que los elementos 1st y 4th están incluidos en la mochila. Para representar toda la población de cromosomas utilizamos una matriz tridimensional (cromosomas [*Tamaño*][*número de elementos*][2]). El *tamaño* representa el número de cromosomas de una población. La segunda dimensión representa el número de elementos que pueden incluirse en la mochila. La tercera dimensión se utiliza para formar la nueva generación de cromosomas.

Diagrama de flujo del programa principal

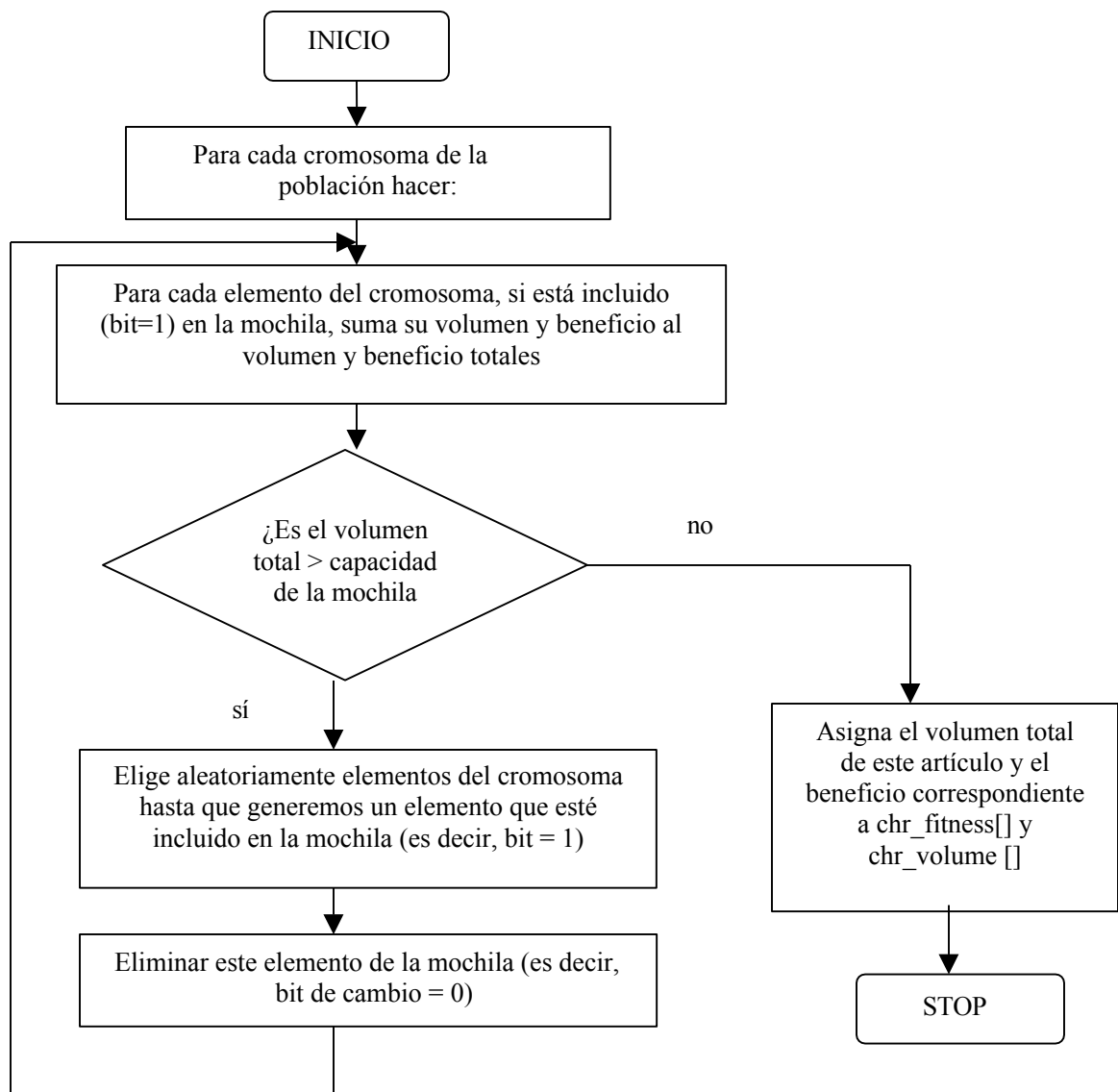


Condiciones de rescisión

La población converge cuando el 90% de los cromosomas de la población tienen el mismo valor de fitness o el número de generaciones es superior a un número fijo.

Función de aptitud

Calculamos la aptitud de cada cromosoma sumando los beneficios de los elementos que se incluyen en la , mientras nos aseguramos de que no se supera la capacidad de la mochila. Si el volumen del cromosoma es mayor que la capacidad de la mochila, se invierte uno de los bits del cromosoma cuyo valor es "1" y se vuelve a comprobar el cromosoma. A continuación se muestra un diagrama de flujo del algoritmo de la función de fitness:

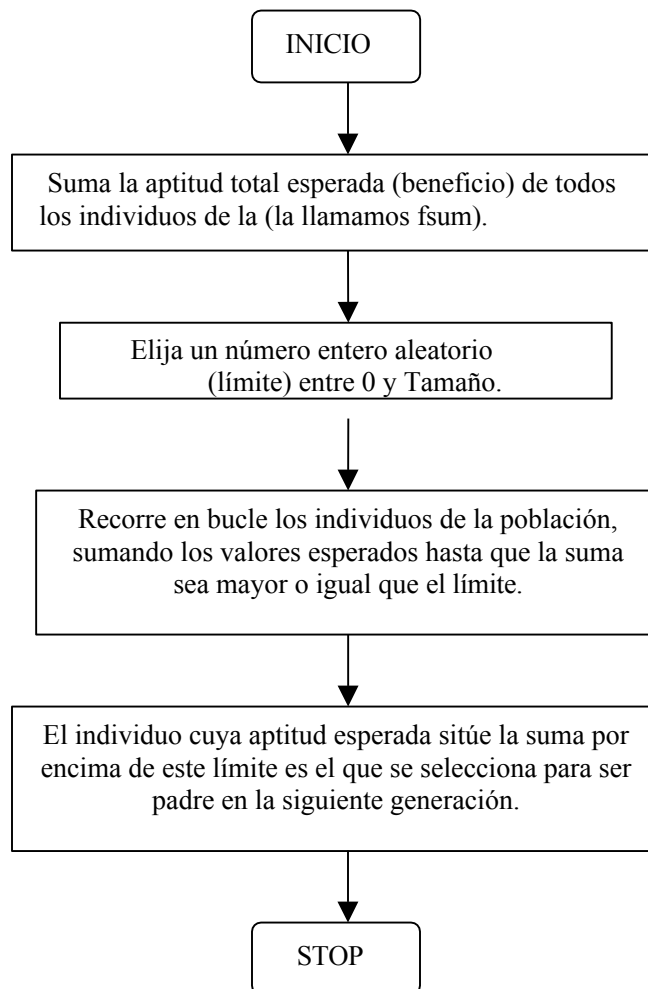


Funciones de selección

En la implementación del programa, probamos dos métodos de selección: la ruleta y la selección de grupos, combinados con el elitismo, en el que dos de los cromosomas más aptos se copian sin cambios a la nueva población, para que no se pierdan las mejores soluciones encontradas.

Selección de ruleta

La ruleta es un método sencillo de aplicar la selección proporcional a la aptitud. Conceptualmente equivale a dar a cada individuo un trozo de una ruleta circular de área igual a la aptitud del individuo [2]. La ruleta se hace girar N veces, siendo N el número de individuos de la población (en nuestro caso $N = \text{Tamaño}$). En cada giro, el individuo bajo el marcador de la rueda es seleccionado para estar en el pool de padres para la siguiente generación [2]. Este método puede aplicarse de la siguiente manera:



Selección de grupos

Implementamos este método de selección para aumentar la probabilidad de elegir cromosomas más aptos para reproducirse más a menudo que cromosomas con valores de aptitud más bajos. Como no encontramos ningún método de selección como éste en la literatura decidimos llamarlo selección de grupo.

Para la implementación del método de selección de grupos, utilizamos otra matriz *indexes[Size]*, donde ponemos los índices de los elementos de la matriz *chr_fitness[Size]*. *chr_fitness*

0	1	2	3	4	5	6	7	8	9	10	11
40	20	5	1	9	7	38	27	16	19	11	3

indices

0	1	2	3	4	5	6	7	8	9	10	11
0	1	2	3	4	5	6	7	8	9	10	11

Ordenamos *los índices* de la matriz en orden descendente según la aptitud de los elementos correspondientes de la matriz *chr_fitness*. Así, los índices de los cromosomas con valores de fitness más altos estarán al principio de los *índices* del array, y los de fitness más bajos estarán hacia el final del array.

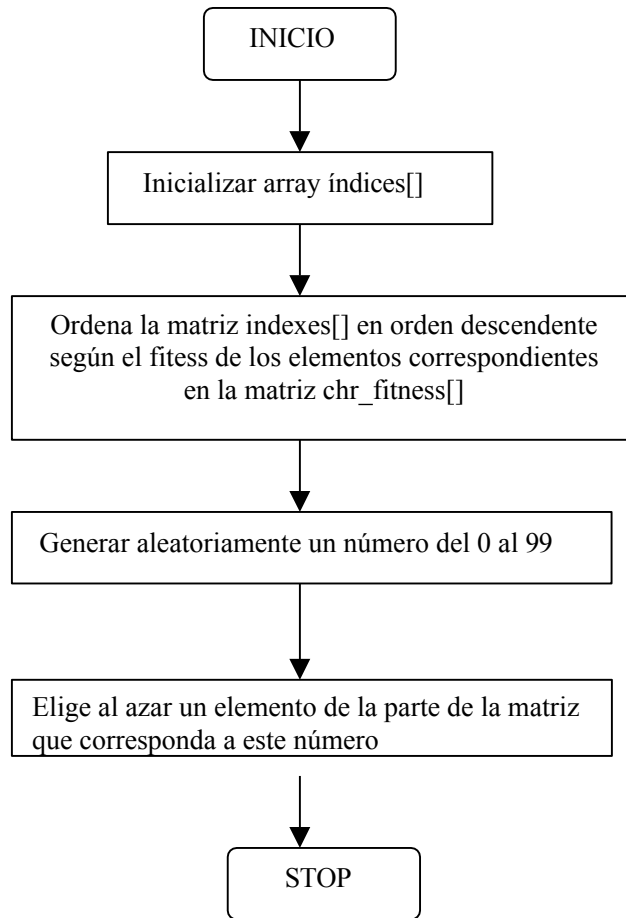
índices:

0	1	2	3	4	5	6	7	8	9	10	11
0	6	7	1	9	8	10	4	5	2	11	3

Dividimos el conjunto en cuatro grupos:

- 0 ... 2 (0 ... Tamaño/4)
- 3 ... 5 (Talla/4 ... Talla/2)
- 6 ... 8 (Talla/2 ... 3*Talla/4)
- 9 ... 11 (3*Talla/4 ... Talla)

A continuación, elegimos al azar un elemento del primer grupo con una probabilidad del 50%, del segundo grupo con una probabilidad del 30%, del tercer grupo con una probabilidad del 15% y del último grupo con una probabilidad del 5%. Así, cuanto más apto sea un cromosoma, más probabilidades tendrá de ser elegido como progenitor en la siguiente generación. A continuación se muestra un diagrama de flujo del algoritmo de selección de grupos:



Comparación entre los resultados de los métodos de ruleta y de selección de grupos

Ratio de cruce= 85%

Ratio de mutación= 0,1%

Sin elitismo

Tamaño de la población	Selección de ruleta			Método de selección de grupos		
	<i>Nº de Gen</i>	<i>Max. Fit. encontrado</i>	<i>Elementos elegidos</i>	<i>Nº de Gen</i>	<i>Max. Fit. encontrado</i>	<i>Elementos elegidos</i>
100	62	2310	2, 5, 8, 13, 15	39	3825	1, 2, 3, 4, 5, 7, 9, 12
200	75	2825	1, 2, 6, 7, 8, 17	51	4310	1, 2, 3, 4, 5, 6, 7, 8, 11
300	91	2825	2, 3, 5, 7, 8, 16	53	4315	1, 2, 3, 4, 5, 6, 7, 8, 10
400	59	2825	1, 2, 3, 4, 15, 16	49	4320	1, 2, 3, 4, 5, 6, 7, 8, 9
500	64	2835	1, 3, 6, 8, 10, 11	65	4320	1, 2, 3, 4, 5, 6, 7, 8, 9
750	97	2840	3, 4, 5, 7, 9, 10	45	4320	1, 2, 3, 4, 5, 6, 7, 8, 9
1000	139	2860	1, 3, 4, 6, 8, 12	53	4320	1, 2, 3, 4, 5, 6, 7, 8, 9

Ratio de cruce= 85% Ratio de
mutación= 0,1%

Elitismo - dos de los cromosomas más aptos se copian sin cambios a una nueva población

Tamaño de la población	Selección de ruleta			Método de selección de grupos		
	<i>Nº de Gen</i>	<i>Max. Fit. encontrado</i>	<i>Elementos elegidos</i>	<i>Nº de Gen</i>	<i>Max. Fit. encontrado</i>	<i>Elementos elegidos</i>
100	60	3845	1, 2, 3, 4, 5, 7, 8, 9	42	3840	1, 2, 3, 4, 5, 6, 7, 12
200	36	3830	1, 2, 3, 5, 6, 7, 8, 10	45	4320	1, 2, 3, 4, 5, 6, 7, 8, 9
250	45	4315	1, 2, 3, 4, 5, 6, 7, 8, 10	45	4315	1, 2, 3, 4, 5, 6, 7, 8, 10
300	46	4320	1, 2, 3, 4, 5, 6, 7, 8, 9	27	4320	1, 2, 3, 4, 5, 6, 7, 8, 9
400	43	4320	1, 2, 3, 4, 5, 6, 7, 8, 9	47	4320	1, 2, 3, 4, 5, 6, 7, 8, 9
500	25	4310	1, 2, 3, 4, 5, 6, 7, 9, 10	54	4320	1, 2, 3, 4, 5, 6, 7, 8, 9
750	34	4315	1, 2, 3, 4, 5, 6, 7, 8, 10	49	4320	1, 2, 3, 4, 5, 6, 7, 8, 9

Ratio de cruce= 75% Ratio de
mutación= 0,1%

Elitismo - dos de los cromosomas más aptos se copian sin cambios a una nueva población

Tamaño de la población	Selección de ruleta			Método de selección de grupos		
	<i>Nº de Gen</i>	<i>Max. Fit. encontrado</i>	<i>Elementos elegidos</i>	<i>Nº de Gen</i>	<i>Max. Fit. encontrado</i>	<i>Elementos elegidos</i>
100	49	3840	1, 2, 3, 4, 6, 7, 8, 9	46	3840	1, 2, 3, 4, 5, 7, 8, 10
200	42	4320	1, 2, 3, 4, 5, 6, 7, 8, 9	43	4315	1, 2, 3, 4, 5, 6, 7, 8, 10
250	39	3820	1, 2, 3, 4, 6, 8, 9, 11	41	4320	1, 2, 3, 4, 5, 6, 7, 8, 9
300	52	4320	1, 2, 3, 4, 5, 6, 7, 8, 9	57	4320	1, 2, 3, 4, 5, 6, 7, 8, 9
400	42	4315	1, 2, 3, 4, 5, 6, 7, 8, 10	45	4320	1, 2, 3, 4, 5, 6, 7, 8, 9
500	44	4320	1, 2, 3, 4, 5, 6, 7, 8, 9	61	4320	1, 2, 3, 4, 5, 6, 7, 8, 9
750	29	4320	1, 2, 3, 4, 5, 6, 7, 8, 9	50	4320	1, 2, 3, 4, 5, 6, 7, 8, 9

Ratio de cruce= 95% Ratio de
mutación= 0,1%

Elitismo - dos de los cromosomas más aptos se copian sin cambios a una nueva población

Tamaño de la población	Selección de ruleta			Método de selección de grupos		
	<i>Nº de Gen</i>	<i>Max. Fit. encontrado</i>	<i>Elementos elegidos</i>	<i>Nº de Gen</i>	<i>Max. Fit. encontrado</i>	<i>Elementos elegidos</i>

ión						
100	27	3320	1, 2, 3, 5, 9, 10, 13	37	3825	1, 2, 3, 4, 5, 6, 9, 13
200	49	4320	1, 2, 3, 4, 5, 6, 7, 8, 9	50	4320	1, 2, 3, 4, 5, 6, 7, 8, 9
250	62	4320	1, 2, 3, 4, 5, 6, 7, 8, 9	40	4310	1, 2, 3, 4, 5, 6, 7, 9, 10
300	31	4320	1, 2, 3, 4, 5, 6, 7, 8, 9	60	4320	1, 2, 3, 4, 5, 6, 7, 8, 9
400	38	4315	1, 2, 3, 4, 5, 6, 7, 8, 10	50	4320	1, 2, 3, 4, 5, 6, 7, 8, 9
500	25	4315	1, 2, 3, 4, 5, 6, 7, 8, 10	56	4320	1, 2, 3, 4, 5, 6, 7, 8, 9
750	34	4315	1, 2, 3, 4, 5, 6, 7, 8, 10	54	4320	1, 2, 3, 4, 5, 6, 7, 8, 9

Los resultados de las dos funciones de selección, ruleta y selección de grupos, difieren mucho dependiendo de si utilizamos el elitismo para mejorar su rendimiento o no.

Cuando no utilizamos el elitismo, el método de selección por grupos es mejor que el método de selección de rueda de ruleta, porque con la selección por grupos la probabilidad de elegir el mejor cromosoma sobre el peor es mayor que con el método de selección de rueda de ruleta. Así, la nueva generación estará formada por cromosomas más aptos y tendrá más posibilidades de aproximarse a la solución óptima.

Cuando utilizamos el elitismo los resultados de la ruleta y del método de selección de grupos son similares porque con el elitismo las dos mejores soluciones encontradas a lo largo de la ejecución del programa no se perderán.

Cruce

Probamos tanto el cruce de uno como el de dos puntos. Como no había una gran diferencia en los resultados que obtuvimos con ambos métodos, decidimos utilizar el cruce de punto único. El punto de cruce se determina aleatoriamente generando un número aleatorio entre 0 y $num_items - 1$. Realizamos el cruce con una probabilidad determinada. Si la probabilidad de cruce es del 100%, se crea una nueva generación mediante el cruce. Si es 0% entonces la nueva generación se hace con copias exactas de los cromosomas de la población anterior. Decidimos una tasa de cruce del 85% probando el programa con diferentes valores. Esto significa que el 85% de la nueva generación se formará con el cruce y el 15% se copiará en la nueva generación.

Mutación

La mutación se realiza para evitar que los AG caigan en un extremo local. Realizamos la mutación en cada posición de bit del cromosoma con una probabilidad del 0,1%.

Complejidad del programa

Dado que el número de cromosomas de cada generación (*Tamaño*) y el número de generaciones (*Gen_número*) son fijos, la complejidad del programa sólo depende del número de elementos que potencialmente pueden colocarse en la mochila. Utilizaremos las siguientes abreviaturas, N para el número de elementos, S para el tamaño de la población y G para el número de generaciones posibles.

La función que inicializa la matriz de *cromosomas* tiene una complejidad de $O(N)$. Las funciones de aptitud, cruce y mutación también tienen complejidades de $O(N)$. Las complejidades de las dos funciones de selección y de la función que comprueba la condición de terminación no dependen de N (sino del tamaño de la población) y tienen tiempos constantes de ejecución $O(1)$.

Las operaciones de selección, cruce y mutación se realizan en un *bucle for* que se ejecuta S veces. Como S es una constante, la complejidad de todo el bucle es $O(N)$. Por último, todas estas operaciones genéticas se realizan en un *bucle do while*, que se ejecuta como máximo G veces. Dado que G es una constante, no afectará a la complejidad asintótica total del programa. Por lo tanto, la complejidad total del programa es $O(N)$.

Conclusión

Hemos mostrado cómo se pueden utilizar los Algoritmos Genéticos para encontrar buenas soluciones para el Problema de la Mochila 0-1. Los AG reducen la complejidad del KP de exponencial a lineal, lo que permite encontrar soluciones aproximadamente óptimas para un problema NP. Los resultados del programa muestran que la implementación de un buen método de selección y el elitismo son muy importantes para el buen rendimiento de un algoritmo genético.

Referencias

1. Obitko, M. Universidad Técnica Checa (CTU). *IV. Algoritmo genético*. Obtenido el 10 de octubre de 2003 de la World Wide Web: <http://cs.felk.cvut.cz/~xobitko/ga/gaintro.html>
2. Mitchell, M. (1998). *An Introduction to Genetic Algorithms (Introducción a los algoritmos genéticos)*. Massachusetts: The MIT Press.
3. Gossett, E. (2003). *Matemáticas discretas con demostración*. New Jersey: Pearson Education Inc.
4. Weiss, M. A. (1999). *Data Structures & Algorithm Analysis in C++ (Estructuras de datos y análisis de algoritmos en C++)*. EE.UU.: Addison Wesley Longman Inc.
5. LeBlanc, T. Informática en la Universidad de Rochester. *Genetic Algorithms*. Obtenido el 10 de octubre de 2003 desde la World Wide Web: <http://www.cs.rochester.edu/users/faculty/leblanc/csc173/genetic-algs/>

Agradecimientos

Me gustaría agradecer la ayuda de la profesora Lydia Sinapova en preparación de este documento.

Para la aplicación de la función de adecuación hemos utilizado el esquema del diagrama de flujo de la siguiente página [web](http://www.evolutionary-algorithms.com/kproblem.html) <http://www.evolutionary-algorithms.com/kproblem.html>.